Module 06 Lab - Regression and Classification Models**Objective:** To understand the difference between regression and classification and to build your first linear models for both tasks.**In this lab, you will write the code to train and evaluate the models.**

Part 1: Regression vs. ClassificationThis is the most fundamental distinction between types of supervised learning problems.* **Regression:** The goal is to predict a **continuous numerical value**. * *Examples:* Predicting the price of a house, the temperature tomorrow, or the stock price.* **Classification:** The goal is to predict a **discrete category or class label**. * *Examples:* Predicting if an email is spam or not spam, if a flower is a setosa, versicolor, or virginica, or if a customer will churn or not.**In this lab, we will tackle one of each.**

Part 2: Linear Regression**Concept:** Linear Regression is used to predict a continuous value. It works by finding the best-fitting straight line through the data points. The model learns a "slope" (coefficient) for each feature and an "intercept".* **Problem:** We will predict the `Fare` of a Titanic passenger based on their `Age` and `Pclass`.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Load and prepare the data
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')

# For simplicity, we'll drop rows with missing age
df.dropna(subset=['Age'], inplace=True)

# Define features and target
features = ['Age', 'Pclass']
target_reg = 'Fare'
X_reg = df[features]
y_reg = df[target_reg]

# Split the data
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42
)
```

Task 1: Train and Evaluate a Linear Regression Model**Your Task:**1. Create an instance of the `LinearRegression` model.2. Train the model using the training data (`X_train_reg`, `y_train_reg`).3. Make predictions on the test data.4. Evaluate the model using `mean_squared_error`. This metric tells us the average of the squared differences between the predicted and actual values.

```
# --- ENTER YOUR CODE HERE ---

# 1. Create the model instance
lr_model = LinearRegression()

# 2. Train the model
lr_model.fit(X_train_reg, y_train_reg)
```

```
# 3. Make predictions
y_pred_reg = lr_model.predict(X_test_reg)

# 4. Evaluate the model
mse = mean_squared_error(y_test_reg, y_pred_reg)

print(f"Mean Squared Error for Fare Prediction: {mse:.2f}")
print(f"The square root of this is {np.sqrt(mse):.2f}, meaning our model is off by about $48 on average.")
```

```
Mean Squared Error for Fare Prediction: 3364.92
The square root of this is 58.01, meaning our model is off by about $48 on average.
```

Part 3: Logistic Regression**Concept:** Despite its name, Logistic Regression is a **classification** algorithm. It works by calculating the probability that a given input belongs to a certain class. It's one of the most widely used and interpretable classification models.* **Problem:** We will predict whether a passenger `Survived` based on their `Age`, `Pclass`, and `Sex`.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Prepare data for classification
# We need to encode 'Sex' column
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})

# Define features and target
features_cls = ['Age', 'Pclass', 'Sex']
target_cls = 'Survived'
X_cls = df[features_cls]
y_cls = df[target_cls]

# Split the data
X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(
    X_cls, y_cls, test_size=0.2, random_state=42
)
```

Task 2: Train and Evaluate a Logistic Regression Model**Your Task:**1. Create an instance of the `LogisticRegression` model.2. Train the model using the classification training data.3. Make predictions on the test data.4. Evaluate the model using `accuracy_score`.

```
# --- ENTER YOUR CODE HERE ---

# 1. Create the model instance
log_model = LogisticRegression()

# 2. Train the model
log_model.fit(X_train_cls, y_train_cls)

# 3. Make predictions
y_pred_cls = log_model.predict(X_test_cls)

# 4. Evaluate the model
accuracy = accuracy_score(y_test_cls, y_pred_cls)

print(f"Accuracy for Survival Prediction: {accuracy:.2%}")
```

```
Accuracy for Survival Prediction: 74.83%
```

📝 Knowledge Check**Instructions:** Answer the following questions in this markdown cell.1. **In your own words, what is the key difference between a regression problem and a classification problem?**2. **The `LinearRegression` model has an attribute called `.coef_`. After you train

the model, print `lr_model.coef_`. What do these numbers represent?**3. Why did we use `mean_squared_error` to evaluate the regression model but `accuracy_score` for the classification model? Why wouldn't accuracy be a good metric for the fare prediction task?**[

1. **In your own words, what is the key difference between a regression problem and a classification problem?**

Regression predicts a numerical value, such as the fare of a Titanic passenger. Classification predicts a category, such as whether a passenger survived or did not survive.

2. **The `LinearRegression` model has an attribute called `.coef_`. After you train the model, print `lr_model.coef_`. What do these numbers represent?**

   The coefficients represent how much each feature affects the predicted fare. In our model, they show how Age and passenger class influence the predicted fare when the other feature stays the same. A positive coefficient increases the prediction, while a negative coefficient decreases it.

3. **Why did we use `mean_squared_error` to evaluate the regression model but `accuracy_score` for the classification model? Why wouldn't accuracy be a good metric for the fare prediction task?**

   We used mean squared error because it measures the differences between the actual fares and the predicted fares. We used accuracy for classification because it measures how many survival predictions were correct. Accuracy would not be a good metric for fare prediction because fares are numerical values, not categories that can simply be marked correct or incorrect.]**

Module 06 Lab - Regression and Classification Models**Objective:** To understand the difference between regression and classification and to build your first linear models for both tasks.**In this lab, you will write the code to train and evaluate the models.**

Part 1: Regression vs. ClassificationThis is the most fundamental distinction between types of supervised learning problems.* **Regression:** The goal is to predict a **continuous numerical value**. * *Examples:* Predicting the price of a house, the temperature tomorrow, or the stock price.* **Classification:** The goal is to predict a **discrete category or class label**. * *Examples:* Predicting if an email is spam or not spam, if a flower is a setosa, versicolor, or virginica, or if a customer will churn or not.**In this lab, we will tackle one of each.**

Part 2: Linear Regression**Concept:** Linear Regression is used to predict a continuous value. It works by finding the best-fitting straight line through the data points. The model learns a "slope" (coefficient) for each feature and an "intercept".* **Problem:** We will predict the `Fare` of a Titanic passenger based on their `Age` and `Pclass`.

```
# Import the required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Load the Titanic dataset
df = pd.read_csv(
    "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
)

# Remove rows with missing Age values
df = df.dropna(subset=["Age"]).copy()

# Define the input features and target
features = ["Age", "Pclass"]
target_reg = "Fare"

X_reg = df[features]
y_reg = df[target_reg]

# Split the data into training and testing sets
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg,
    y_reg,
    test_size=0.2,
     random_state=42
)

# Display the dataset and split sizes
print("Dataset shape:", df.shape)
print("Training rows:", len(X_train_reg))
print("Testing rows:", len(X_test_reg))
```

```
Dataset shape: (714, 12)
Training rows: 571
Testing rows: 143
```

Task 1: Train and Evaluate a Linear Regression Model**Your Task:**1. Create an instance of the `LinearRegression` model.2. Train the model using the training data (`X_train_reg`, `y_train_reg`).3. Make predictions on the test data.4. Evaluate the model using `mean_squared_error`. This metric tells us the average of the squared differences between the predicted and actual values.

```
# 1. Create the model instance
lr_model = LinearRegression()

# 2. Train the model
lr_model.fit(X_train_reg, y_train_reg)

# 3. Make predictions on the test data
y_pred_reg = lr_model.predict(X_test_reg)

# 4. Evaluate the model
mse_reg = mean_squared_error(y_test_reg, y_pred_reg)

print("Mean Squared Error:", mse_reg)
```

```
Mean Squared Error: 3364.922912856981
```

The Linear Regression model was trained to predict passenger fares using Age and Pclass. The dataset was divided into training and testing sets, with 571 records used for training and 143 for testing. The model produced a Mean Squared Error of approximately 3,364.92. This metric measures the average squared difference between the predicted fares and the actual fares. The result provides a baseline for evaluating the model's prediction performance.

Part 3: Logistic Regression**Concept:** Despite its name, Logistic Regression is a **classification** algorithm. It works by calculating the probability that a given input belongs to a certain class. It's one of the most widely used and interpretable classification models.* **Problem:** We will predict whether a passenger `Survived` based on their `Age`, `Pclass`, and `Sex`.

```
# Import the required classification tools
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Encode Sex as numerical values
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})

# Define the features and target
features_cls = ['Age', 'Pclass', 'Sex']
target_cls = 'Survived'

X_cls = df[features_cls]
y_cls = df[target_cls]

# Split the classification data
X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(
    X_cls,
    y_cls,
    test_size=0.2,
    random_state=42
)

# Display the resulting dataset sizes
print("Classification dataset shape:", df.shape)
print("Training rows:", len(X_train_cls))
print("Testing rows:", len(X_test_cls))
```

```
Classification dataset shape: (714, 12)
Training rows: 571
Testing rows: 143
```

Task 2: Train and Evaluate a Logistic Regression Model**Your Task:**1. Create an instance of the `LogisticRegression` model.2. Train the model using the classification training data.3. Make predictions on the test data.4. Evaluate the model using `accuracy_score`.

```
# 1. Create the Logistic Regression model
log_model = LogisticRegression(max_iter=1000)

# 2. Train the model
log_model.fit(X_train_cls, y_train_cls)

# 3. Predict survival on the test data
y_pred_cls = log_model.predict(X_test_cls)

# 4. Evaluate the model using accuracy
accuracy = accuracy_score(y_test_cls, y_pred_cls)

print("Classification Accuracy:", accuracy)
```

```
Classification Accuracy: 0.7482517482517482
```

```
# Display the learned Linear Regression coefficients
print("Linear Regression coefficients:", lr_model.coef_)
```

```
Linear Regression coefficients: [ -0.43695889 -38.29728935]
```

📝 Knowledge Check**Instructions:** Answer the following questions in this markdown cell.1. **In your own words, what is the key difference between a regression problem and a classification problem?**2. **The `LinearRegression` model has an attribute called `.coef_`. After you train the model, print `lr_model.coef_`. What do these numbers represent?**3. Why did we use `mean_squared_error` to evaluate the regression model but `accuracy_score` for the classification model? Why wouldn't accuracy be a good metric for the fare prediction task?**

1. What is the key difference between a regression problem and a classification problem?

Regression is used to predict a continuous numerical value, such as the fare a passenger paid on the Titanic while classification is used to predict a category or class, such as whether a passenger survived or did not survive.

2. What do the numbers in lr_model.coef_ represent?

The numbers represent how each input feature affects the predicted fare when the other is held constant. In this model, the coefficient for Age is approximately -0.437, while the coefficient for Pclass is approximately -38.297. That is according to the fitted model, an additional year of age is associated with a decrease of about 0.437 in predicted fare, holding passenger class constant. A one-unit increase in Pclass is associated with a decrease of about 38.297 in predicted fare, holding age constant. These are model-estimated relationships, not proof of cause and effect.

3. Why did we use mean_squared_error for regression but accuracy_score for classification? Why would accuracy not be a good metric for fare prediction?

We used Mean Squared Error (MSE) for regression because it measures the average squared difference between the actual and predicted numerical fares. We used accuracy for classification because it measures the proportion of survival predictions that match the actual survival outcomes. Accuracy would not be suitable for fare prediction because fares are continuous numerical values, not categories. MSE helps us measure how far the predicted fares are from the actual fares.

Module 06 Lab - Regression and Classification Models**Objective:** To understand the difference between regression and classification and to build your first linear models for both tasks.**In this lab, you will write the code to train and evaluate the models.**

Part 1: Regression vs. ClassificationThis is the most fundamental distinction between types of supervised learning problems.* **Regression:** The goal is to predict a **continuous numerical value**. * *Examples:* Predicting the price of a house, the temperature tomorrow, or the stock price.* **Classification:** The goal is to predict a **discrete category or class label**. * *Examples:* Predicting if an email is spam or not spam, if a flower is a setosa, versicolor, or virginica, or if a customer will churn or not.**In this lab, we will tackle one of each.**

Part 2: Linear Regression**Concept:** Linear Regression is used to predict a continuous value. It works by finding the best-fitting straight line through the data points. The model learns a "slope" (coefficient) for each feature and an "intercept".* **Problem:** We will predict the `Fare` of a Titanic passenger based on their `Age` and `Pclass`.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
# Load and prepare the datadf = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')
# For simplicity, we'll drop rows with missing agedf.dropna(subset=['Age'], inplace=True)
# Define features and targetfeatures = ['Age', 'Pclass']target_reg = 'Fare'X_reg = df[features]y_reg = df[target_reg]
# Split the dataX_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(X_reg, y_reg, test_size=0.2, random_state=4
```

Task 1: Train and Evaluate a Linear Regression Model**Your Task:**1. Create an instance of the `LinearRegression` model.2. Train the model using the training data (`X_train_reg`, `y_train_reg`).3. Make predictions on the test data.4. Evaluate the model using `mean_squared_error`. This metric tells us the average of the squared differences between the predicted and actual values.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

df = pd.read_csv(
    'https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv'
)

df.dropna(subset=['Age'], inplace=True)

features = ['Age', 'Pclass']
target_reg = 'Fare'

X_reg = df[features]
y_reg = df[target_reg]

X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42
)
```

```
lr_model = LinearRegression()

lr_model.fit(X_train_reg, y_train_reg)

y_pred_reg = lr_model.predict(X_test_reg)

mse = mean_squared_error(y_test_reg, y_pred_reg)

print(f"Mean Squared Error for Fare Prediction: {mse:.2f}")
print(f"Root Mean Squared Error: {np.sqrt(mse):.2f}")
```

```
Mean Squared Error for Fare Prediction: 3364.92
Root Mean Squared Error: 58.01
```

Part 3: Logistic Regression**Concept:** Despite its name, Logistic Regression is a **classification** algorithm. It works by calculating the probability that a given input belongs to a certain class. It's one of the most widely used and interpretable classification models.* **Problem:** We will predict whether a passenger `Survived` based on their `Age`, `Pclass`, and `Sex`.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})

features_cls = ['Age', 'Pclass', 'Sex']
target_cls = 'Survived'
X_cls = df[features_cls]
y_cls = df[target_cls]

X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(X_cls, y_cls, test_size=0.2, random_state=42)
```

Task 2: Train and Evaluate a Logistic Regression Model**Your Task:**1. Create an instance of the `LogisticRegression` model.2. Train the model using the classification training data.3. Make predictions on the test data.4. Evaluate the model using `accuracy_score`.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

log_model = LogisticRegression(max_iter=1000)

log_model.fit(X_train_cls, y_train_cls)

y_pred_cls = log_model.predict(X_test_cls)

accuracy = accuracy_score(y_test_cls, y_pred_cls)

print(f"Accuracy for Survival Prediction: {accuracy:.2%}")
```

```
Accuracy for Survival Prediction: 74.83%
```

📝 Knowledge Check**Instructions:** Answer the following questions in this markdown cell.1. **In your own words, what is the key difference between a regression problem and a classification problem?**2. **The `LinearRegression` model has an attribute called `.coef_`. After you train the model, print `lr_model.coef_`. What do these numbers represent?**3. **Why did we use `mean_squared_error` to evaluate the regression model but `accuracy_score` for the classification model?** Why wouldn't accuracy be a good metric for the fare prediction task?

1. What is the main difference between regression and classification problems?

Regression predicts continuous numerical values, such as house prices or ticket prices. Classification predicts categories or labels, such as whether a passenger survived. In simple terms, regression predicts numbers, while classification predicts categories.

2. What does `.coef_` represent in the `LinearRegression` model?

`.coef_` represents the coefficients of each feature. In this experiment, they correspond to Age and Pclass. They show how each feature affects the predicted fare when the other feature stays constant.

3. Why do regression models use `mean_squared_error` while classification models use `accuracy_score`? Why is accuracy unsuitable for predicting ticket prices?

MSE measures the difference between predicted and actual prices, while accuracy measures the percentage of correct classifications. Accuracy is not suitable for predicting ticket prices because prices are numerical values, not categories. MSE is better for measuring how close the predicted prices are to the actual prices.